

To Ban or not to Ban? How Open Source Projects Govern GenAI Contributions

Wenhao Yang
Peking University
Beijing, China
yangwh@stu.pku.edu.cn

Runzhi He
Peking University
Beijing, China
rzhe@pku.edu.cn

Minghui Zhou*
Peking University
Beijing, China
zhmh@pku.edu.cn

Abstract

Generative AI (GenAI) is playing an increasingly important role in open source software (OSS). Beyond completing code and documentation, GenAI is increasingly involved in issues, pull requests, code reviews, and security reports. Yet, cheaper generation does not mean cheaper review — and the resulting maintenance burden has pushed OSS projects to experiment with GenAI-specific rules in contribution guidelines, security policies, and repository instructions, even including a total ban on AI-assisted contributions. However, governing GenAI in OSS is far more than a ban-or-not question. The responses remain scattered, with neither a shared governance framework in practice nor a systematic understanding in research. Therefore, in this paper, we conduct a multi-stage analysis on various qualitative materials related to GenAI governance retrieved from 67 highly visible OSS projects. Our analysis identifies recurring concerns across contribution workflows, derives three governance orientations, and maps out 12 governance strategies and their policy instruments. We show that governing GenAI in OSS extends well beyond banning — it requires coordinated responses across accountability, verification, review capacity, code provenance, and platform infrastructure. Overall, our work distills dispersed community practices into a structured overview, providing a conceptual baseline for researchers and a practical reference for maintainers and platform designers.

CCS Concepts

• **Software and its engineering** → **Open source model**; *Software development process management*; *Collaboration in software development*; Empirical software validation.

Keywords

generative AI, open source software, contribution workflows, contribution governance

1 Introduction

Generative AI is actively reshaping the future of OSS communities. It is increasingly acting as a community contributor—submitting issues, pull requests, code reviews, and security reports [14, 24, 73]. Recent coding agents take this further by navigating repositories, generating full patches, and executing pipelines with greater autonomy [34, 64, 71]. With the rise of GenAI, how developers enter, participate in, and evolve community collaboration is undergoing a fundamental shift.

However, the unprecedented speed of generating contributions comes at a cost. A prosperous and sustainable open source community depends not only on active contribution, but also on the

capacity to review, triage, and maintain incoming work [1, 11, 21, 36, 57, 65, 76]. As a maintainer of matplotlib observed, AI “changes the cost balance between generating and reviewing code” [39]. The surge of AI generated contributions grew beyond the maintenance capabilities of many well-known open source projects, causing chaos and debates within the cornerstones of the modern digital infrastructure. The asymmetry can be stark: when a Node.js TSC member submitted a 19,000-line PR generated with Claude Code [7], the incident triggered a petition by over 80 developers [26, 27] calling for a project-wide ban on AI-assisted contributions.

In response, projects have begun to establish GenAI-specific rules across contribution guidelines, security policies, issue and PR templates, AI directives (such as AGENTS.md), and public announcements. The range of these responses is wide: some projects categorically reject AI-generated contributions on provenance and licensing grounds; others require disclosure of AI use and impose accountability conditions on contributors; still others constrain AI tool behavior directly through repository-level instructions; and some have gone further, suspending external PRs or even migrating their primary collaboration venue. At the same time, many projects acknowledge the productivity benefits and recognize that “continuously rejecting LLM-generated code is not a good approach” [25]. These diverse responses suggest that governing¹ GenAI in OSS extends well beyond a binary question of permitting or prohibiting AI—yet current practices remain scattered across heterogeneous governance channels, with no shared framework or established best practice in sight.

This gap is both practical and academic. Prior research has extensively studied GenAI’s technical capabilities across software engineering tasks [14, 24, 73], its adoption and effects on developer productivity and OSS collaboration [2, 16, 40, 49, 74], and the long-standing governance mechanisms of OSS contribution management [11, 21, 57, 76]. However, these streams have not yet converged on the question of how OSS projects govern GenAI within contribution workflows. We still lack a systematic understanding of what concerns GenAI raises across different stages of OSS contribution workflows, and how projects organize their responses into governance approaches and concrete strategies.

In this paper, we present the first systematic analysis of GenAI governance in OSS. We collected repository-embedded governance materials (e.g., CONTRIBUTING, SECURITY, issue and PR templates, AGENTS.md) and surrounding justificatory materials (e.g., announcements, discussion threads) from 67 highly visible OSS

*Corresponding author.

¹Following prior work on OSS contribution governance [1, 11, 12], we use *governance* here to refer to the rules, procedures, and collaboration interfaces through which projects regulate contribution intake, review, accountability, and maintenance.

projects. Through iterative qualitative coding, we identified recurring concerns, governance orientations, and cross-project strategies. Specifically, we answer the following research questions:

RQ1. *What concerns does GenAI raise across OSS contribution workflows?* **Result:** We identified seven recurring concern areas distributed across code contribution, communication, issue entry, security reporting, incentive structures, provenance and licensing, and platform infrastructure.

RQ2. *What governance approaches do OSS projects develop for GenAI contributions?* **Result:** We derived three governance orientations: prohibitionist, boundary-and-accountability, and quality-first—operationalized through 12 recurring governance strategies.

Our main contributions in this paper are:

- We provide a structured account of how GenAI-related concerns are distributed across OSS contribution workflows;
- We derive three governance orientations that explain why projects facing similar GenAI pressures develop markedly different institutional responses;
- We abstract a reusable strategy space of 12 governance strategies and their policy instruments, showing how the strategies serve different roles under different orientations.

Overall, our work offers both a conceptual baseline for future research on GenAI and software collaboration, and a practical reference for maintainers, communities, and platform designers seeking to establish or evaluate their own governance approaches.

2 Related Work

In this section, we review prior work on: 1) GenAI for software engineering; 2) the adoption and influence of GenAI in the scope of OSS; 3) how OSS communities review, moderate, and maintain contributions.

2.1 GenAI for Software Development

Since the rise of large language models, we have observed and measured the effect of this technology in various software engineering tasks, including code generation, testing, repair, and broader development assistance [14, 24, 73]. Researchers have evaluated the code quality and found limitations of code completion products like GitHub Copilot [47]. More recently, researchers have shifted focus to coding agents (e.g., SWE-Agent [71], OpenHands [64]) that navigate repositories, generate full patches, and execute pipelines with greater autonomy. Recent OSS-facing work also examines context-engineering practices and AGENTS.md files used to provide repository-specific instructions to such agents [20, 43]. Through large-scale mining of agent-assisted pull requests, researchers [34, 64, 71] investigate the status and draw the technical boundaries of GenAI in software development.

2.2 GenAI Use and Impact in Software and OSS Collaboration

Researchers have also noted the influences of GenAI in other dimensions of software engineering, specifically in developer productivity, software quality, and collaboration. Prior studies observed significant productivity gain from the use of GitHub Copilot [49], along with higher validation and coordination costs in realistic development settings [2, 17, 40]. Surveys of AI coding assistants

in practice report broad adoption, but also persistent trust and workflow concerns [55]. In OSS contexts, Xiao et al. [68] examine self-admitted GenAI usage patterns across OSS repositories and, as a secondary contribution, qualitatively analyze 13 policy documents from 12 projects, classifying each by its observable stance toward GenAI as prohibitive, restrictive, or supportive; related studies analyze how developers self-declare AI-generated code and what such self-declarations contain [32, 61]. Other recent work has further explored vibe coding, Cursor AI, auto-merged agentic PRs, and human-AI review practices, revealing implications for vulnerability exposure, complexity, code survival, and review dynamics [4, 16, 18, 23, 72, 74]. Related work also begins to surface compliance and provenance risks around AI-generated code, especially license uncertainty and compatibility in OSS settings [67, 70], as well as tool support for provenance and license analysis of generated code [3]. Overall, this stream explains how GenAI is used and what effects it has on software work and OSS collaboration.

2.3 OSS Contribution Governance, Maintainer Workload, and Moderation

Our study builds most directly on prior work on OSS contribution governance and maintainer workload. Pull-based development, PR review, triage, and newcomer barriers have long been recognized as central issues in OSS collaboration [21, 57, 76]. More fine-grained studies have examined the governance interfaces through which projects manage these pressures: contribution guidelines often diverge from actual contribution processes [12], issue templates shape the processability of incoming reports [59], and testing-related guidance in contribution documents remains uneven across projects [13]. Related work has also shown that issue triage, maintainer scalability, duplicate submissions, review styles, automation infrastructure, and security reporting mechanisms are not merely coordination details, but important governance mechanisms for filtering inputs, allocating attention, and sustaining order in OSS projects [1, 11, 31, 35, 36, 65, 69, 75]. This literature provides an important baseline for our work: the significance of GenAI lies not only in accelerating content generation, but also in intensifying and reshaping long-standing OSS problems around input filtering, accountability, review capacity, and maintainer overload.

2.4 The Knowledge Gaps

Existing work has mapped GenAI’s technical capabilities, developer adoption patterns, and individual productivity gains, while also establishing the importance of contribution governance and maintainer burden in traditional OSS. The closest prior work is Xiao et al. [68], who study self-admitted GenAI usage in OSS and, as part of that broader analysis, classify 13 policy documents from 12 projects as prohibitive, restrictive, or supportive toward GenAI use. This provides an important first view of observable policy stance, but it leaves open a broader governance question: how do OSS projects define the GenAI problem, where do they encode governance rules, and through what mechanisms do they regulate GenAI-mediated contribution workflows? Our study addresses this question by analyzing 67 OSS projects across contribution guidelines, security policies, issue and PR templates, AGENTS.md files, and surrounding discussions. Rather than coding only a document’s stance toward

GenAI, we identify the concerns that motivate governance, derive the orientations that organize project responses, and abstract the concrete strategies through which those responses are operationalized. Going beyond previous findings on projects’ positions toward AI contribution, our orientations encode the rationale from perception to action. The distinction from previous stances is especially sharp in the quality-first orientation (O3). O3 projects reassert existing quality, reviewability, and maintainer-capacity thresholds instead of designing AI-specific rules — not “supportive”.

3 Methodology

3.1 Research Design and Scope

We conducted a comparative qualitative document analysis of publicly visible governance materials that OSS projects use to regulate GenAI-mediated contribution workflows. We did not aim to estimate policy prevalence or test a predefined theory. Instead, we sought to explain how OSS projects articulate GenAI-related problems, organize them into broader governance orientations, and express them through concrete governance strategies.

We therefore focused on *publicly encoded governance*: texts that explicitly connect GenAI-related inputs to rules about admissibility, disclosure, accountability, verification, review, reporting, sanctions, or infrastructural arrangements. We treated a text as evidence of GenAI governance only when it explicitly regulated GenAI-assisted contribution behavior rather than merely mentioning AI, automation, or code quality. We fixed this inclusion criterion before document analysis began, building on prior definitions of OSS contribution governance [1, 11, 12]. The criterion stayed stable throughout screening; we only refined how it applied to borderline cases, so it was not shaped by our findings. Inclusion did not depend on source type: whether a rule appears in a wiki page, a blog post, or a markdown file explicitly marked for AI governance did not affect the decision. A source qualified when it was project-linked, maintainer-authored, or directly referenced by sampled governance materials and stated explicit rules for handling GenAI-assisted contributions; a CONTRIBUTING file or issue thread that merely mentioned AI did not.

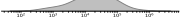
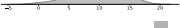
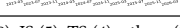
Document analysis fits this goal because contribution guides, security policies, templates, AI policies, and tool-facing instructions are visible collaboration interfaces rather than post hoc commentary. The corpus comprises 67 OSS projects: 58 repositories and 9 communities. The communities are project-adjacent policy sources at the foundation, organization, or ecosystem level that directly govern OSS contribution practices. For consistency, we use the term “OSS project” for both kinds of cases throughout the paper.

3.2 Corpus Construction

To combine broad initial coverage with theoretically motivated follow-up sampling, we constructed the corpus in two stages.

Phase 1: broad repository screening. We created the seed frame from the top 800 starred repositories listed on Gitstar Ranking’s repository leaderboard [19]. This choice gave us a high-visibility set of projects with relatively mature governance surfaces and active external contribution workflows. We first removed non-code repositories (e.g., awesome lists and documentation-only collections)

Table 1: Demographics of the sampled GitHub projects.

Statistics	Mean	Median	Distribution
# of Stars	49,197.95	45,304.00	
# of Commits	45,523.98	15,540.00	
# of Contributors	1,666.12	1,045.50	
Age (years)	9.86	10.03	
Policy adoption	2025-07	2025-11	
Language	Python (10), Go (8), C++ (8), JS (5), TS (4), others (20)		

from the candidate pool, so no non-engineered repository entered screening. Two authors then independently examined each remaining repository’s governance surfaces (CONTRIBUTING, SECURITY, standalone AI policies, AGENTS.md, and issue/PR templates) for public text that tied GenAI-related inputs to contribution rules, review expectations, security handling, channel restrictions, or sanctions. General references to AI, automation, or code quality were excluded unless they were explicitly connected to OSS contribution governance. Disagreements were resolved through discussion. This stage yielded 37 OSS projects with explicit GenAI governance content.

Phase 2: snowball expansion. We then expanded the corpus through structured snowballing, following the iterative procedure formalized by Wohlin [66]. Starting from the 37 seed projects, we retrieved and read the commits, issues, and PRs that added or modified the governance rules. For each relevant external reference—third-party policy pages, foundation governance texts, or AI policies of other projects—we manually validated it against the inclusion criterion and decided whether to expand the corpus. Along these paths we also captured governance surfaces underrepresented in the seed set, such as standalone AI policies, AI-specific security reporting rules, AGENTS.md files, and platform-migration announcements. Materials that merely mentioned AI without setting contribution rules were excluded at every stage. Expansion stopped when two criteria were met: *connectivity saturation*, when new links no longer produced qualifying cases, and *category saturation*, when new cases no longer introduced new codes. We deliberately opted out of random sampling because explicit GenAI governance rules remain rare in the open source world, and snowballing from a well-defined seed set is an established procedure for finding rare, interconnected artifacts. This stage added 30 cases and brought the final corpus to 67 OSS projects: 58 repositories and 9 communities. As shown in Table 1, the sampled repositories are highly visible, well-established, and collaboratively intensive.

3.3 Data Sources and Units of Analysis

For each OSS project, we collected up to two kinds of public materials. The primary evidence consisted of *repository-embedded governance materials*: README files, CONTRIBUTING files, GOVERNANCE files, SECURITY files, issue and PR templates, relevant files under .github/, standalone AI policy pages, and tool-facing files such as AGENTS.md or Copilot instructions.

The second kind of material consisted of *surrounding justification materials*, such as maintainer-authored announcements, policy-related commit messages, linked issue or discussion threads, and

project-linked external policy pages. Commits, issues, and PRs thus played two roles in our analysis: as snowballing paths when they introduced or modified GenAI rules (Section 3.2), and as context for interpreting the associated governance texts. In coding, these materials were treated as contextual evidence rather than standalone policy types: orientation assignments and strategy coding were anchored in stable governance text and only refined through justificatory materials when the two were consistent.

We analyzed the latest stable public version of each core governance text available as of February 2026 and used earlier discussions or commits to interpret policy formation and adoption context. For each project, we recorded the URL, access date, source type, governance surfaces, and the versioned location of the core policy text when available in a corpus sheet used as an audit trail.

Our analysis operated at two levels. At the project level, each OSS project formed a project-level policy case, consolidating all relevant documents from that project into a single unit, ensuring that the same stance expressed across multiple files was not counted as separate policies. At the text level, we coded strategy-bearing text segments: passages that expressed rules or judgments about admissibility, disclosure, responsibility, verification, provenance, reporting channels, sanctions, or infrastructural adjustment.

3.4 Coding Procedure

We analyzed the corpus through iterative qualitative coding with constant comparison, following a provisional coding approach [41] in five steps: (1) drafting an initial codebook from community discussions and technical blog posts; (2) stabilizing it on an exploratory subset through independent coding by two authors; (3) independently coding the full corpus with the refined codebook; (4) computing inter-rater agreement; and (5) adjudicating remaining disagreements through joint re-review, with the third author resolving cases where the two coders still disagreed. The three stages below describe the analytical layers produced by this process.

Stage 1: open coding of maintainer concerns. Two authors independently conducted open coding on an initial heterogeneous subset of OSS projects to develop a shared codebook for recurring maintainer concerns. We then used the refined codebook to independently code the full corpus.

Stage 2: project-level memoing and governance orientations. After concern coding became stable, we wrote project-level memos summarizing each OSS project’s main governance problem framing and relevant governance surfaces. On this basis, we assigned each OSS project a primary governance orientation by comparing its dominant governance logic across documents. Orientation was treated as a project-level interpretive abstraction rather than a document label. When a project displayed mixed signals, we re-reviewed the relevant texts jointly and assigned orientation based on the dominant governance judgment while retaining mixed or escalatory elements in the strategy analysis.

Stage 3: strategy abstraction and implementation analysis. We then compared coded OSS projects across the corpus to abstract cross-project governance strategies. We retained a strategy only

when it was analytically distinct in governance function, behavioral target, and policy instruments. Throughout this stage, we repeatedly moved between project-level cases, coded segments, and emerging categories to refine strategy boundaries and clarify how strategies played different roles across governance orientations.

To strengthen analytic credibility, two authors independently performed both seed screening and full-corpus coding. Inter-rater agreement was computed at the case level after the codebook had stabilized (step 4) and before adjudication (step 5). For governance orientation, which assigned one dominant label to each of the 67 project-level cases, Cohen’s kappa was 0.742. For concern and strategy coding, which allowed multiple codes per case, we calculated Cohen’s kappa per code on binary presence/absence decisions and report the macro-average across codes: 0.745 for concerns and 0.840 for strategies. All concern and strategy counts reported below are case-level presence counts rather than mention frequencies, so repeated expressions within the same case were counted once. Most orientation disagreements clustered at the O2/O3 boundary. For the same OSS project, we compared multiple governance surfaces whenever possible and revisited borderline cases during adjudication. The full codebook is part of our replication package. For every concern, orientation, and strategy code, it gives an operationalized definition, rules on when to apply the code, and boundary notes distinguishing it from adjacent codes. The category descriptions in Sections 4 and 5 are condensed from these definitions.

4 RQ1: Maintainer Concerns

Our corpus shows that GenAI enters OSS workflows well beyond code generation, including code contribution, discussion, issue triage, security reporting, and platform-level entry points. Maintainer concerns therefore span seven recurring concern areas: 1) review bottlenecks and contributor responsibility in code contribution, 2) low-value AI text in communication, 3) rising triage costs at issue entry, 4) pressure on high-priority security channels, 5) adversarial incentives, 6) copyright/licensing/provenance uncertainty, and 7) platform defaults and tooling limits.

1. Code contribution. In code contribution, maintainers most consistently describe a *review bottleneck*. GenAI makes PRs easier to generate without making them easier to review, explain, or validate. This concern appears in four closely related manifestations:

- (1) **Intent and context.** A first recurring pressure is that AI makes it easier to submit changes whose scope outruns shared problem definition. Docusaurus warns that projects now receive “1k LOC PRs that are obviously AI-generated and implement unsolicited features” [10]. It also reminds contributors that significant changes require prior issue-based approval. The problem is not PR size alone, but that maintainers are pushed to decide during review whether the change should exist at all.
- (2) **Surface completeness and correctness.** A second recurring concern is that surface completeness no longer signals correctness. AI can produce polished, plausible-looking changes with test coverage that still fail at the semantic level. The Metasploit project described this pattern bluntly: some AI-assisted submissions were “well-formatted, looked really neat, and did

nothing it said it did” [53]. Review therefore cannot shift from correctness checking to style checking.

- (3) **Authorship and responsibility.** A third concern is whether a contributor actually understands, explains, and owns the change. typescript-eslint captures this especially sharply: maintainers do not want review to become “effectively babysitting someone’s Claude instance via the code review process” [63]. The issue is not AI use alone, but whether a human author remains accountable.
- (4) **Review capacity.** A fourth concern is that review capacity becomes the first bottleneck. Projects therefore describe AI contribution primarily as a review problem rather than a generation problem. Oh My Zsh summarizes the asymmetry succinctly: “review is the bottleneck. Not code generation.” [54]. FastAPI makes the same point even more forcefully, describing low-effort AI submissions as “a Denial-of-service attack on our human effort” [15].

2. Communication. In communication and discussion, the same responsibility problem appears as *low-value AI text* and weaker human exchange. typescript-eslint states this directly: AI-generated PR descriptions are often “nearly-verbatim descriptions of the code diff, which add no value to the PR” [62]. Jaeger states the corresponding expectation for review interaction just as succinctly: “Code review is a discussion between people, not bots” [29]. The objection is therefore not simply to verbosity, but to generated text that adds little context and displaces direct human exchange.

3. Issue triage. At issue entry, the problem is *triage costs*: AI makes reports look plausible without making them more actionable. NewPipe encodes this problem explicitly by banning AI-written issue and PR templates, stating that “Those texts are often lengthy and miss critical information” [46]. What matters at this stage is whether a report contains the specific information needed for triage.

4. Security reporting. Security reporting is the same triage problem in *high-priority channels*. curl states this concern particularly clearly: because security reports are investigated with priority, “fake and otherwise made up security problems effectively prevent us from doing real project work and make us waste time and resources” [8]. Fabricated or unverifiable AI reports are especially costly because they immediately consume scarce security-handling time.

5. Adversarial incentives. In some contexts, maintainers frame the problem as adversarial incentives. When rewards remain in place, AI lowers the cost of producing bad-faith or low-value reports. curl provides the clearest example in the context of bug bounties: “A bug bounty gives people too strong incentives to find and make up ‘problems’ in bad faith that cause overload and abuse” [58]. Here the concern is not mere noise, but noise made worthwhile by institutional incentives.

6. Provenance and licensing. For some projects, the concern is not output quality but copyright/licensing/provenance *uncertainty*. QEMU states the legal uncertainty directly: with AI content generators, “the copyright and license status of the output is ill-defined with no generally accepted, settled legal foundation” [52]. NetBSD

operationalizes the same concern even more bluntly by treating LLM output as “presumed to be tainted code” [45]. Here the issue is not ordinary review quality, but whether such inputs can be admitted at all.

7. Platform and tooling. Finally, some projects locate part of the problem in *platform defaults/tooling limits* that amplify the concerns above. Zig’s maintainers describe GitHub’s Copilot issue composer as taking a user’s “simple explanation of a bug” and rewriting it into “a multi-paragraph monologue with no interesting details and added false information” [77]. tldraw links its temporary closure of external PRs directly to infrastructure limits, calling it “a temporary policy until GitHub provides better tools for managing contributions” [60]. Here the issue is not only what contributors do, but what platforms make easy and what projects still cannot control.

5 RQ2: Governance Orientations and Strategies

To answer RQ2, we first identify three governance orientations that explain why projects facing similar GenAI pressures develop markedly different institutional responses. We then derive 12 concrete governance strategies through which these orientations are operationalized across collaboration surfaces.

5.1 Governance orientations

After open-coding maintainer concerns, we compared how projects interpreted these problems, where they located core risks, and how they allocated governance responsibility. We found that cross-project variation cannot be explained simply in terms of whether a project “allows AI.” Instead, through iterative comparison, we derived three recurring governance orientations—higher-order judgments, not static labels or maturity rankings—that reflect three distinct ways of framing the governance problem: whether certain inputs should enter the workflow at all; whether they may enter under explicit boundaries of accountability, visibility, and verification; or whether they should be absorbed into existing quality and maintainer-cost controls. Organizational form and domain context function as boundary conditions that shape where pressure first arises, but do not reliably predict which orientation a project adopts. Across the 67 cases, O2 was the most common dominant orientation (40 cases, 59.7%), followed by O1 (14, 20.9%) and O3 (13, 19.4%). Twenty-three cases displayed mixed governance logics, combining a dominant orientation with material elements of a secondary one.

O1: Prohibitionist / zero-tolerance. The core judgment behind O1 is that certain GenAI-related inputs present *structural, non-absorbable risk*—including provenance uncertainty, copyright and licensing incompatibility, or taintedness—and should therefore *not* be absorbed into the existing review process. The prior question is not how to review such inputs more effectively, but whether they should enter the workflow at all. QEMU states that it will “DECLINE any contributions which are believed to include or derive from AI generated content” [52]. NetBSD similarly states that LLM-generated code “is presumed to be tainted code, and must not be committed without prior written approval by core” [45]. These texts define AI-generated code as a class of input whose admissibility is in question from the outset.

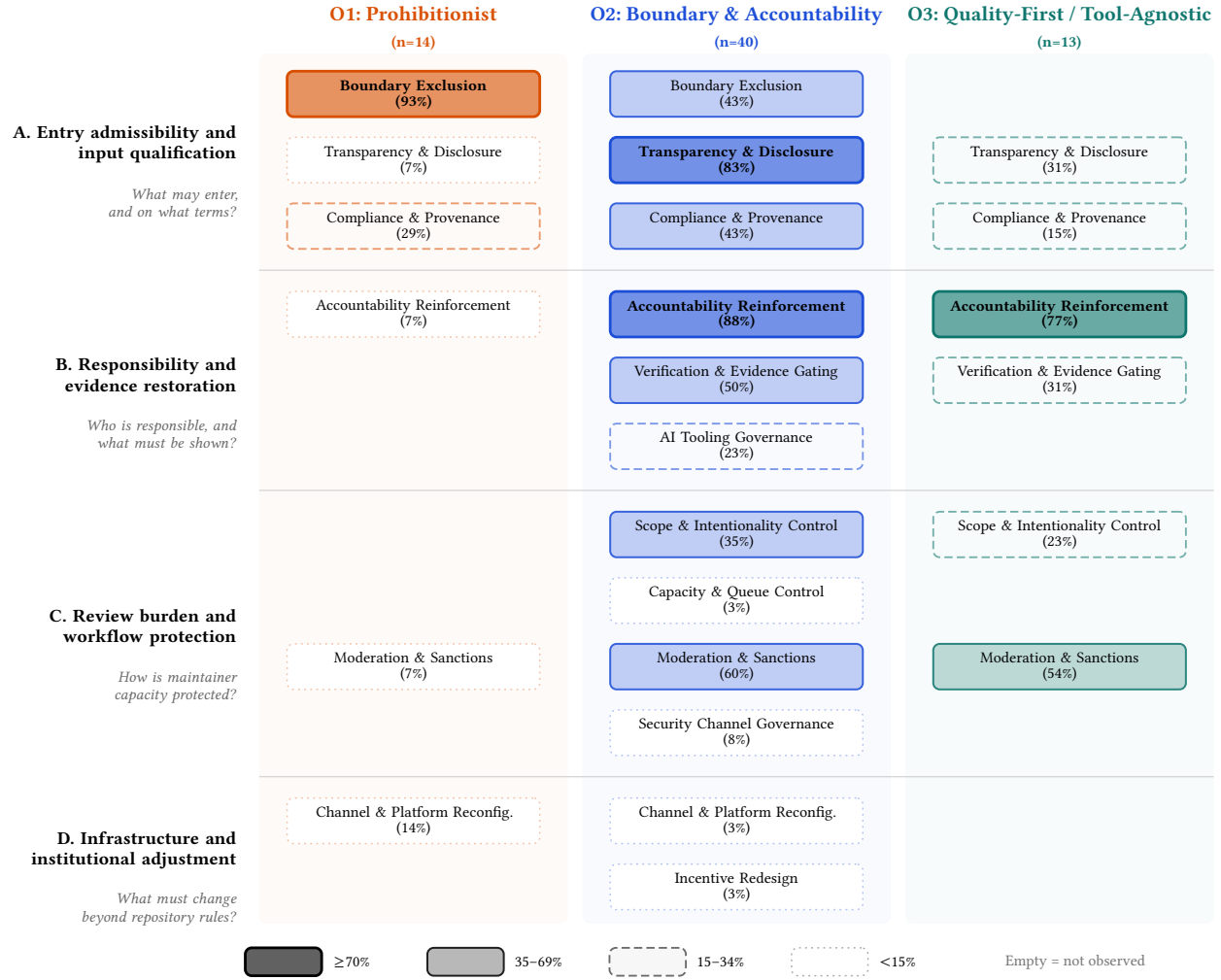


Figure 1: Governance strategies mapped by functional group (rows) and governance orientation (columns). Shading intensity reflects within-orientation prevalence using four bands ($\geq 70\%$, 35–69%, 15–34%, and $< 15\%$). Percentages in parentheses show the share of cases within that orientation that exhibit the strategy, while orientation totals are reported in the column headers. Empty cells indicate that the strategy was not observed in that orientation.

O2: Boundary-and-accountability. The core judgment behind O2 is that GenAI-related inputs may enter the workflow, but only under explicit conditions of accountability, visibility, and verification. Rather than excluding AI-mediated input categorically, this orientation treats AI as a distinct governance object requiring explicit rules across multiple collaboration surfaces (*multi-surface boundary governance*). CloudNativePG states that the goal is “not to ban AI entirely,” but to shift “the burden of proof back to the contributor,” requiring “full human accountability for any AI-assisted work” [6]. llama.cpp draws the boundary more explicitly: AI tools may be used “solely in an assistive capacity,” and “all AI usage requires explicit disclosure” [37]. O2 thus locates the governance problem in AI-mediated input itself and admits such input only under AI-specific rules of disclosure, human accountability, verification, or enforcement.

O3: Quality-first / tool-agnostic. The core judgment behind O3 is that projects prefer to absorb AI-related inputs into existing expectations around contribution quality, maintainability, and maintainer cost, rather than building AI-specific governance rules (*reasserting existing quality thresholds under cheaper-generation conditions*). The key question is not whether AI was used, but whether the input is reviewable, maintainable, and worth maintainers’ limited time. Oh My Zsh expresses this clearly: “Not a moral stance. Not a ban. Just clarity around expectations and accountability” and “We’re not interested in policing tooling. We’re interested in accountability.” [54]. curl articulates the same logic: licensing and quality obligations apply “independent if AI is used or not” [8]. O3 thus locates the problem not in AI use per se, but in ordinary contribution quality under cheaper-generation conditions, addressed through general controls of reviewability, testing, scope, and maintainability.

Boundary conditions. The boundaries among the three orientations lie in higher-order judgments about what kind of governance problem GenAI presents, not in any single device such as disclosure or templates. The O2/O3 boundary was the least sharp. Our tie-breaker was whether a project introduced AI-specific admissibility or accountability boundaries (O2), or instead reasserted quality-first review controls that apply regardless of AI use (O3). The llama.cpp and curl policies quoted above illustrate the difference: llama.cpp admits AI use only under mandatory disclosure, whereas curl applies the same obligations whether or not AI is used. Orientations are dominant case-level logics, and strategies can co-occur with any orientation. Under O1, for example, disclosure supports narrow exceptions or enforcement rather than contradicting prohibition. The three orientations should therefore be read as ideal-typical governance logics rather than static labels or a ranking of policy strictness.

5.2 Governance strategies

While governance orientations capture *why* projects respond differently, governance strategies capture *how* those orientations are operationalized across contribution workflows. Through cross-project comparison, we derived 12 strategies that cluster into four functional groups: *entry admissibility and input qualification*, *responsibility and evidence restoration*, *review burden and workflow protection*, and *infrastructure and institutional adjustment*. Each strategy takes effect through *policy instruments*, the textual and workflow mechanisms a project deploys. Typical examples include disclosure checkboxes and template fields, evidence requirements, PR limits, security-channel rules, tool-facing instruction files, and platform configuration changes. Figure 1 maps their *within-orientation prevalence*. In broad terms, O1 concentrates on front-gate and infrastructural controls, O2 carries the broadest AI-specific governance layer, and O3 relies more heavily on general-purpose quality controls while adopting fewer explicit AI-specific mechanisms.

Function A. Entry admissibility and input qualification.

These strategies move governance toward the point of intake, addressing what kinds of inputs may enter the system and what qualifying information must accompany them.

A1. Boundary Exclusion / Reject-or-Ban.

Boundary exclusion governs the behavior of *excluding certain classes of GenAI-mediated input from the acceptable contribution surface altogether*. Unlike disclosure, accountability, or verification, this strategy does not ask how to review GenAI-related input more effectively. It asks a prior admissibility question: whether such input should be treated as reviewable at all. Accordingly, its clearest expressions are not requests for explanation, but statements such as “will not accept,” “does not allow,” “decline,” or “must not be committed.”

As illustrated by QEMU’s and NetBSD’s explicit exclusion language (see O1 above), boundary exclusion operates not as a stricter review rule, but as a refusal to treat certain inputs as ordinarily admissible.

The main benefit is front-loading the forms of uncertainty maintainers most want to avoid at the stage of admissibility judgment, reducing the need to repeatedly re-litigate boundaries during review.

The cost is equally clear: projects narrow the surface of acceptable participation. Boundary exclusion is near-universal in O1 (93%) but also appears in 43% of O2 projects, where it typically applies selectively to specific contribution surfaces—such as fully generated PRs or unverified security reports—rather than as a blanket prohibition.

A2. Transparency and Disclosure.

Disclosure seeks not to determine whether AI may be used, but to turn AI use from a hidden practice into a visible, discussable, and reviewable input signal. Its immediate target is contributors’ reporting behavior when submitting issues, PRs, comments, or reports. What maintainers need is not abstract transparency for its own sake, but a way to calibrate expectations once inputs become longer, more polished, and more contribution-like. Disclosure helps them distinguish between what appears to come from the contributor’s own understanding and what depends heavily on model-generated output.

Spec Kit states that if a contributor uses “any kind of AI assistance,” it “must be disclosed in the pull request or issue,” and adds that failure to disclose makes it harder “to determine how much scrutiny to apply” [56]. This captures disclosure not as moral signaling, but as review calibration. In more formalized settings, the same logic is pushed into templates or dedicated fields so that AI use becomes intake information rather than an after-the-fact suspicion. A few projects additionally request documentation of prompts or AI interactions. We observed this too rarely for a standalone strategy, but it works as a policy instrument supporting disclosure, provenance, or evidence gating.

The main benefit of disclosure is that it reduces maintainers’ up-front information-discovery cost. They do not need to first infer how an input was produced before deciding how to process it. At the same time, the trade-off is concrete: contributors must bear additional metadata-reporting costs, and explicit AI marking may generate concerns about differential treatment. For this reason, many projects pair disclosure requirements with explicit statements that they do not intend to police tools per se, but to use disclosure as a more rational review-calibration signal.

A3. Compliance and Provenance Safeguarding.

Unlike boundary exclusion, compliance and provenance safeguards do not necessarily require direct prohibition. More often, they appear in projects that allow some degree of AI use while insisting that contributors retain responsibility for redistributability, license compatibility, and lawful provenance. The central behavior being governed is therefore the contributor’s obligation to make these judgments before submission, rather than leaving them for maintainers to reconstruct case by case during review.

mpv states that AI-assisted contributions “are not forbidden,” but the submitter “takes full responsibility” and must confirm that the code can be submitted under the relevant license; curl similarly insists that “the burden is on them to ensure no unlicensed code is submitted” [8, 44]. The core move is to push provenance and redistributability checks back onto contributors before technical review begins.

The benefit is reallocating legal and provenance uncertainty away from maintainers. The cost falls on contributors, who must evaluate not only whether the content works but whether it can

be redistributed under clear provenance. This strategy’s role varies sharply by orientation: in O1, provenance uncertainty serves as a rationale for categorical exclusion; in O2, it becomes a contributor-facing compliance obligation; in O3, it is folded into general licensing responsibility.

Function B. Responsibility and evidence restoration.

These strategies address how projects restore accountability, explanation, and evidence where GenAI makes them thinner.

B1. Accountability Reinforcement.

Accountability reinforcement seeks to pull responsibility back from the tool to the named human contributor. It does not primarily regulate whether AI was used. Instead, it regulates whether the submitter understands the content, can explain it during review, and is willing to remain responsible for revising, defending, and maintaining it. Accountability indicates answerability and ownership in review, not blame. In this way, the strategy redefines authorship as a responsibility relation rather than a mere act of submission.

CloudNativePG states that “The human contributor is the sole party responsible for the contribution,” and that “The AI generated it” is “never an acceptable answer”; Jaeger echoes the same expectation with “You own everything you submit” and “Code review is a discussion between people, not bots” [5, 29]. These policies treat authorship less as the act of opening a PR than as the continuing duty to explain, revise, and stand behind it.

The principal benefit is reducing maintainers’ uncertainty about who really understands the change and will remain responsible for it. The trade-off is substantial: this strategy raises the bar for participation by tying admissibility to explainability and sustained responsibility. In O2, accountability reinforcement forms part of an explicit AI-specific boundary regime (“The AI generated it is never an acceptable answer”), whereas in O3 it is more often absorbed into general contribution-quality expectations.

B2. Verification and Evidence Gating.

Verification and evidence gating seeks to change contributor behavior by requiring sufficient supporting material before an input is processed further. This may include local test logs, proof-of-concept artifacts, reproduction steps, references, or explicit human validation notes. The key governance move is not to ask reviewers to be more careful, but to shift the burden of demonstrating verifiability toward the submitter.

Kornia encodes verification as an explicit gate: “Every PR introducing functional changes must include a pasted snippet of the local test logs,” and implementations must cite an “existing library reference” for verification [33]. In security reporting, similar hard evidence thresholds apply (see Security Channel Governance below). These rules shift the burden from reviewer inference to submitter-provided evidence.

The benefit is reducing secondary verification cost during review; the trade-off is a clear increase in submission preparation cost. Verification is more prevalent in O2 (50%) than O3 (31%), reflecting O2’s tendency to add evidence requirements as part of its AI-specific

boundary regime, whereas O3 projects more often rely on pre-existing quality gates without adding new AI-motivated evidence thresholds.

B3. AI Tooling Governance.

AI tooling governance belongs in this group because it also seeks to restore responsibility and control, but does so at the tool layer rather than only at the human layer. Compared with accountability and verification, it is a further step: projects do not merely regulate how humans should use AI, but begin to regulate AI tools and agents directly through files such as `AGENTS.md`, Copilot instructions, or other tool-facing repository rules.

JabRef’s `AGENTS.md` makes the tool-facing nature of this strategy explicit: agents must “Never commit generated code without human review,” must not “Write entire PRs,” and must not “Automate the submission of code changes” [28]. The result is governance that addresses not only human conduct, but the expected behavior of the tools themselves.

The benefit of tooling governance is that it allows some governance work to happen *before* low-quality or poorly attributable inputs reach review, by constraining the tool’s role directly. It therefore shifts some control from *ex post* evaluation to *ex ante* interface design. Its costs are also distinctive. First, it depends heavily on whether tools and agent ecosystems actually read and comply with repository-level instructions. Second, projects themselves must maintain a layer of policy text written for machine consumption and keep it aligned with ordinary contributor-facing rules. The challenge here is therefore not only rule design, but the stability of the interface between repository policy and tool behavior.

Function C. Review burden and workflow protection.

These strategies protect maintainer time, review queues, and high-priority channels once inputs have already entered the system.

C1. Scope and Intentionality Control.

Scope and intentionality control governs not only PR size, but whether a contribution is framed within the right problem context. Projects use this strategy to require small, focused, issue-linked, and intentionally scoped submissions, thereby discouraging broad, speculative, or misaligned PRs that arrive before the underlying problem has been accepted.

CloudNativePG states “Design First” for non-trivial changes and warns that PRs arriving “out of the blue” may be closed if they do not align with project context [5]. PyTorch formalizes the same logic upstream by stating that “PRs must have an associated actionable Issue” [51]. In both cases, the point is to ensure that review does not become the place where maintainers must first reconstruct why the work should exist.

The benefit of this strategy is that it lowers the cost of problem redefinition during review. Maintainers are less often forced to ask why a contribution exists, why it takes its current scope, and whether it fits the project’s priorities. The trade-off is that more filtering work moves upstream into issue triage and proposal stages, reducing the space for drive-by contributions and fast exploratory participation.

C2. Capacity and Queue Control.

Capacity and queue control treats the review queue itself as a scarce resource in need of governance. Instead of focusing on the technical content of an individual submission, it regulates input rate, concurrency, and review load. This strategy makes explicit a practical reality of OSS maintenance: even when contributors act in good faith, review bandwidth remains limited, and GenAI can significantly increase the rate at which reviewable-looking inputs are produced.

Jaeger states that “we limit the number of simultaneous open PRs for new contributors” and that excess PRs may be labeled “on-hold” or closed [30]. Here the governance target is not the correctness of any single patch, but how much simultaneous reviewer attention a contributor may occupy. The benefit is making maintainer time governable as a resource; the cost is that waiting time and participation friction are reallocated to contributors, especially newcomers.

C3. Moderation and Sanctions.

Moderation and sanctions govern what happens once certain classes of input have already been judged low-effort, repeatedly non-compliant, adversarial, or not worth further processing. Rather than trying to improve such inputs, this strategy defines more consistent ways to close, reject, block, or ban them, thereby reducing the cost of repeated case-by-case negotiation.

pandas states that maintainers may “ban users from contributing” if they violate the guidelines repeatedly; CPython likewise notes that unproductive AI-generated issues or PRs may be closed and repeat submitters “may be blocked”; curl goes further in security contexts: “We ban users immediately who submit made up fake reports” [8, 48, 50]. Moderation is widespread in both O2 and O3 but functions less as a standalone governance logic than as an enforcement amplifier whose effectiveness depends on other strategies—disclosure, verification, or scope control—already being in place.

C4. Security Channel Governance.

Security channel governance belongs in this group because it protects a particularly scarce and high-value workflow: vulnerability handling. Unlike ordinary PRs or general issue reports, security submissions often receive elevated processing priority. Security channel governance also differs from the general evidence gating of B2. B2 gates ordinary PR and issue handling, whereas C4 protects high-priority security channels, where fake reports (fabricated, hallucinated, or unverifiable vulnerability reports) immediately consume scarce incident-response time. This strategy therefore governs what counts as an actionable security input and what level of evidence, disclosure, and reproducibility is required before high-priority maintainer resources are engaged.

curl warns that it is “rarely a good idea to just copy and paste an AI generated report” and instead tells reporters to “write the report yourself” after verifying the problem [8]. llama.cpp adds a hard evidence threshold by requiring that “Your report must include a working Proof-of-Concept” [38]. The shared logic is to front-load actionability before scarce security attention is engaged.

The benefit of this strategy is that it prevents fabricated, speculative, or unverifiable inputs from crowding out real vulnerabilities. The trade-off is a higher reporting threshold: weaker but potentially

useful leads may be filtered out if they do not meet the required standard of evidence. Projects thus exchange lower waste in security triage for a narrower and more demanding reporting interface.

Function D. Infrastructure and institutional adjustment.

When repository-level rules are insufficient, some projects adjust the surrounding infrastructure or institutional conditions under which contributions enter the system.

D1. Channel, Venue, and Platform Reconfiguration.

Channel, venue, and platform reconfiguration governs not primarily *what* contributors submit, but *where* and *through which interface* they are allowed to submit it. When maintainers conclude that repository-level rules can no longer control the relevant problems, they may suspend external PRs, redirect reporting channels, or even migrate their main collaboration venue.

ldraw states that it is “not accepting pull requests from external contributors” and that such PRs “will be automatically closed” until GitHub provides better tools for managing contributions [60]. Zig represents a stronger infrastructural move: after migrating to Codeberg, it explicitly expected “fewer violations” of its strict no-AI policy because GitHub had been pushing AI-assisted entry points so aggressively [78]. Here the governance response is not to review inputs differently, but to change the channel through which they arrive.

The benefit of this strategy is that it changes the entry architecture itself rather than asking maintainers to repeatedly interpret and enforce the same boundaries after the fact. Its costs, however, are substantial. Contributors must adapt to new channels or platforms, and projects assume new infrastructural burdens and potential ecosystem friction. For this reason, reconfiguration typically appears as an escalation strategy when repository-level governance is judged insufficient.

D2. Incentive Redesign.

Incentive redesign is one of the least frequent but most revealing strategies in the corpus. It does not regulate how a particular input is written or reviewed. Instead, it targets the broader reward structure that makes certain kinds of harmful input worth producing in the first place. In this sense, it governs not content quality directly, but the institutional conditions that generate persistent low-value or adversarial submissions.

curl makes this logic unusually explicit: it removed rewards “to remove the incentives for submitting made up lies” and later generalized that “A bug bounty gives people too strong incentives to find and make up problems in bad faith” [58]. By addressing harmful input at the level of institutional reward rather than downstream filtering, this strategy functions as a high-cost governance escalation that may also discourage some good-faith participation.

6 Threats to Validity and Limitations

6.1 Coverage and Transferability

Our corpus intentionally privileges visible OSS projects and policy-rich governance environments. The seed frame began with the top-ranked repositories on Gitstar Ranking’s repository leaderboard and was then extended through theoretically motivated snowball

sampling, resulting in a varied and reasonably sized corpus. Accordingly, the findings should be interpreted as an analytically transferable account of publicly visible GenAI governance rather than a prevalence estimate for the entire OSS ecosystem. Smaller projects, low-activity repositories, non-GitHub communities, and non-English communities may encode similar pressures less explicitly or govern them differently.

6.2 Publicly Encoded Governance vs. Enacted Practice

This study analyzes governance as it is publicly expressed and institutionally encoded in project materials. Actual governance may also depend on private maintainer discussions, discretionary triage, informal coordination, moderation practices, or security workflows that are not visible in repository texts. Our findings should therefore be read as an account of public governance interfaces rather than a complete account of every path through which OSS projects actually process AI-related contributions. We also do not directly evaluate the causal effectiveness of these rules or whether they are enforced uniformly. Our results are coded from public texts and have not been validated by the repository owners or maintainers, so our interpretations of individual policies may contain imprecisions. Most analyzed materials are maintainer-authored or directly linked from the projects’ own governance texts, so this limitation is unlikely to invalidate the descriptive taxonomy, though it does bound what we can claim about intent, enforcement, and outcomes. Maintainer interviews and surveys remain important future work.

6.3 Interpretive and Temporal Limits

These categories remain interpretive abstractions rather than naturally given classes. The three orientations are best understood as ideal-typical dominant logics, and orientation counts should therefore be read as approximate patterning rather than sharp natural groupings, especially near the O2/O3 boundary. Strategy findings are less sensitive to that boundary because strategy presence was coded independently of orientation.

In addition, GenAI-related governance is changing quickly: platforms, tools, and project rules continue to shift over short periods of time. Our analysis is thus a cross-sectional account of one time window, and later governance changes may alter how particular OSS projects are best interpreted.

7 Discussion

7.1 GenAI Reshapes and Front-loads OSS Governance

Our results demonstrate that GenAI governance is rarely a from-scratch invention. Instead, it represents a front-loading redesign of existing governance mechanisms. Pull-based development has always faced “review friction” [21], and the open source communities have invented strategies, including contribution and security guidelines, issue and PR templates, proposal and approval workflows, to gate external contributions, balance maintainer workload, and coordinate collaboration efforts.

GenAI creates a radical socio-technical asymmetry, where the cost of producing a plausible, complete-looking PR approaches

zero, while the cost of reviewing it remains high or even increases. Consequently, we observe a shift from *downstream code review* to *upstream admission control*. Strategies like *Scope and Intentionality Control* (requiring pre-approved issues) or *Verification and Evidence Gating* (requiring proof-of-concept before review) are attempts to protect the most scarce resource in OSS: maintainer attention [11]. By forcing contributors to justify the “why” and “how” before maintainers look at the “what,” projects are effectively moving the triage line further up the workflow to survive the “denial-of-service attack” of AI-assisted inputs.

7.2 From Repository to Infrastructure-Level Governance

While most projects in the corpus have shown agility in updating governance policies in markdown files, our results reveal the limits of repository-level governance. Text-based rules (CONTRIBUTING, SECURITY) are often reactive and might easily be bypassed by coding agents or low-effort human submitters. The platform-and-tooling concern identified in RQ1 suggests that as long as platforms provide a “one-click” AI generation path without corresponding identification or intake controls, repository rules will remain under siege.

We are beginning to see the emergence of infrastructure-level countermeasures that bypass the repository ruleset entirely. For example, Vouch [22] reframes entry as a web-of-trust problem, where contributions are filtered based on endorsement before review: participants may need to be vouched for before interacting with selected project surfaces, and projects can extend this further into a web-of-trust model across repositories. Similarly, Agent-Trace [9] proposes a machine-readable provenance format of AI contributions, conversation links, and file- or line-level attribution, to shift the burden of “proof of work” from human disclosure to tool-level transparency. These shifts suggest that the future of OSS governance lies not just in writing better markdown files, but in reconfiguring the infrastructure itself to make GenAI contributions legible and governable by default.

7.3 Implications for Maintainers and Platform Designers

For *maintainers*, the immediate implication is that there is no “one-size-fits-all” AI policy. Our taxonomy reads as a *diagnosis-to-strategy map* (Figure 1): provenance or licensing risk points to Boundary Exclusion (A1) or Compliance and Provenance Safeguarding (A3); review overload points to Scope and Intentionality Control (C1), Verification and Evidence Gating (B2), or Capacity and Queue Control (C2); security-channel noise points to Security Channel Governance (C4); and agent- or platform-driven bypass points to AI Tooling Governance (B3) or Platform Reconfiguration (D1). Each path carries an explicit trade-off: stricter intake controls protect maintainer capacity but raise participation costs and may discourage external contribution, so projects should move up the map gradually rather than adopt maximal controls by default. The O2/O3 distinction matters for practice. Treating a review-throughput problem as an AI-specific one leads to over-engineering—new AI gates where existing quality controls suffice—whereas treating a provenance or accountability problem as generic quality control leads to

under-engineering. Identifying the right orientation tells maintainers which family of strategies to use first, protecting the project without blocking valuable human-AI collaboration.

For *platform designers*, the focus must shift from “generation power” to “intake control” and “traceability.” Shipping features that encourage “effortless-to-produce but expensive-to-validate” contributions without providing the tools to detect, prioritize, or block them creates an unsustainable burden on the community. Platforms must facilitate higher-information collaboration interfaces—such as machine-readable AI metadata—to ensure that the productivity gains of GenAI do not come at the cost of OSS community burnout.

7.4 Future Work

Efficacy of Disclosure Policies. While *Transparency and Disclosure* is a dominant and seemingly approachable strategy, little is known about its actual compliance rate. Future work may empirically measure the effectiveness of disclosure rules – specifically, how often AI use is self-admitted versus hidden, and whether mandatory disclosure leads to systemic bias in how maintainers treat certain contributions.

Automated Triage. As the volume of agent-assisted PRs increases, maintainers need proactive tools to manage the “attention tax” [42]. Researchers may develop triage methods to identify low-quality agentic contributions with possibly high maintenance cost, enabling maintainers to fast-fail costly “ghosting” attempts.

Long-term Code and Community Health. The rise of agentic coding may translate into a significant “flow-debt tradeoff”. He et al. [23] find that the adoption of Cursor can transiently boost development velocity, while leading to a persistent increase in static analysis warnings and code complexity. Future longitudinal studies are needed to assess whether GenAI governance regimes successfully maintain long-term software quality, or if they inadvertently accelerate technical debt.

8 Conclusion

This paper provides a systematic account of how OSS projects govern GenAI in contribution workflows. Drawing on a corpus of public governance materials from 67 OSS projects, we showed how GenAI enters OSS collaboration at multiple workflow stages, what maintainer concerns it raises across these stages, and what governance responses projects articulate.

Our results show that GenAI governance in OSS is not reducible to the question of “to ban or not to ban.” Instead, projects confront governance challenges across the full contribution lifecycle – spanning code reviewability, communication accountability, triage burden, security-channel integrity, provenance uncertainty, and platform constraints. We identified three governance orientations (prohibitionist, boundary-and-accountability, and quality-first) that explain how projects interpret these challenges differently, and abstracted 12 cross-project governance strategies and their policy instruments.

Taken together, these findings reveal a structural shift in OSS governance — from code review toward upstream admission control, and from text-based policies toward infrastructure-level assurance. For maintainers, the right governance path begins with diagnosing

the bottleneck. Platform designers must invest in intake control and traceability to match the ease of generation.

References

- [1] Adam Alami, Raúl Pardo, Marisa Leavitt Cohn, and Andrzej Wasowski. 2022. Pull request governance in open source communities. *IEEE Transactions on Software Engineering* 48, 12 (2022), 4838–4856. doi:10.1109/TSE.2021.3128356
- [2] Matthew Baird, Mar Carpanelli, Brian Xu, and Kevin Xu. 2024. Early evidence on the impact of generative AI on software engineers’ employment outcomes. LinkedIn Economic Graph. <https://economicgraph.linkedin.com/blog/early-evidence-on-the-impact-of-generative-ai-on-software-engineers-employment-outcomes> Accessed: 2026-03-26.
- [3] Daniele Bifulco, Guido Annicchiarico, Pierluigi Barbiero, Massimiliano Di Penta, and Fiorella Zampetti. 2025. CodeGenLink: A Tool to Find the Likely Origin and License of Automatically Generated Code. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 3993–3996. doi:10.1109/ASE63991.2025.00358
- [4] Ruben Branco, Paulo Canelas, Catarina Gamboa, and Alcides Fonseca. 2026. LGTM! characteristics of auto-merged LLM-based agentic PRs. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR ’26)*. Association for Computing Machinery, New York, NY, USA, 1–5. <https://pcanelas.com/assets/papers/2026-msr-lgtm.pdf> Accepted at MSR 2026 Mining Challenge; preprint available online. Accessed: 2026-03-26.
- [5] CloudNativePG Contributors. 2026. CloudNativePG AI Contribution Policy. GitHub repository file. https://github.com/cloudnative-pg/governance/blob/main/AI_POLICY.md Accessed: 2026-03-26.
- [6] CloudNativePG Contributors. 2026. Proposal: Adoption of Official AI Contribution & Copyright Policy. GitHub Issue. <https://github.com/cloudnative-pg/governance/issues/45> Accessed: 2026-03-26.
- [7] Matteo Collina. 2026. Virtual File System for Node.js. GitHub Pull Request #61478, nodejs/node. <https://github.com/nodejs/node/pull/61478> Accessed: 2026-03-26.
- [8] curl Contributors. 2026. On AI use in curl. GitHub repository file. <https://github.com/curl/curl/blob/master/docs/CONTRIBUTE.md> Accessed: 2026-03-26.
- [9] Cursor Contributors. 2026. Agent Trace. Open specification, version 0.1.0 (RFC). <https://agent-trace.dev/> Accessed: 2026-03-27.
- [10] Docusaurus Contributors. 2026. Contributing to Docusaurus: AI-assisted PRs. GitHub repository file. <https://github.com/facebook/docusaurus/blob/main/CONTRIBUTING.md> Accessed: 2026-03-26.
- [11] Nadia Eghbal. 2020. *Working in Public: The Making and Maintenance of Open Source Software*. Stripe Press, South San Francisco, CA, USA.
- [12] Omar Elazhary, Margaret-Anne D. Storey, Neil A. Ernst, and Andy Zaidman. 2019. Do as I do, not as I say: do contribution guidelines match the GitHub contribution process? In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Los Alamitos, CA, USA, 286–290. doi:10.1109/ICSME.2019.00043
- [13] Bruna Faluccci, Felipe Gomide, and André Hora. 2025. What do contribution guidelines say about software testing?. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, Los Alamitos, CA, USA, 434–438. doi:10.1109/MSR66628.2025.00073
- [14] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, Los Alamitos, CA, USA, 31–53. doi:10.1109/ICSE-FOSE59343.2023.00008
- [15] FastAPI Contributors. 2026. Contributing to FastAPI: Automated Code and AI. GitHub repository file. <https://github.com/fastapi/fastapi/blob/master/docs/en/docs/contributing.md#automated-code-and-ai> Accessed: 2026-03-26.
- [16] Ahmed Fawzy, Amjed Tahir, and Kelly Blineoe. 2025. Vibe coding in practice: motivations, challenges, and a future outlook – a grey literature review. arXiv preprint arXiv:2510.00328. doi:10.48550/arXiv.2510.00328 [cs.SE]
- [17] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of developer productivity: there’s more to it than you think. *ACM Queue* 19, 1 (2021), 20–48. doi:10.1145/3454122.3454124
- [18] Haoyu Gao, Peerachai Banyongrakul, Hao Guan, Mansoor Zahedi, and Christoph Treude. 2026. On autopilot? An empirical study of human-AI teaming and review practices in open source. arXiv preprint arXiv:2601.13754. doi:10.48550/arXiv.2601.13754 [cs.SE]
- [19] Gitstar Ranking. 2026. Repositories Ranking - Gitstar Ranking. Website. <https://gitstar-ranking.com/repositories> Accessed: 2026-03-26.
- [20] Thibaud Gloaguen et al. 2026. Evaluating AGENTS.md: are repository-level context files helpful for coding agents? arXiv preprint arXiv:2602.11988. doi:10.48550/arXiv.2602.11988 [cs.SE]

- [21] Georgios Gousios, Martin Pinzger, and Arie van Deursen. 2014. An exploratory study of the pull-based software development model. In *Proceedings of the 36th International Conference on Software Engineering*. Association for Computing Machinery, New York, NY, USA, 345–355. doi:10.1145/2568225.2568260
- [22] Mitchell Hashimoto and Vouch Contributors. 2026. Vouch. GitHub repository. <https://github.com/mitchellh/vouch> Accessed: 2026-03-27.
- [23] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2026. Speed at the cost of quality: how Cursor AI increases short-term velocity and long-term complexity in open-source projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. Association for Computing Machinery, New York, NY, USA, 1–19. <https://arxiv.org/abs/2511.04427> Accepted at MSR 2026 Mining Challenge; preprint available at arXiv. Accessed: 2026-03-26.
- [24] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: a systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–41. doi:10.1145/3695988
- [25] Immich Contributors. 2026. Object storage. GitHub Discussion. <https://github.com/immich-app/immich/discussions/23745> Accessed: 2026-03-26.
- [26] Fedor Indutny. 2026. No AI Code in Node.js Core. Change.org petition. <https://www.change.org/p/no-ai-code-in-node-js-core> Accessed: 2026-03-26.
- [27] Fedor Indutny. 2026. No AI in Node.js Core: A Petition to Disallow Acceptance of LLM-Generated Pull Requests. GitHub Repository. <https://github.com/indutny/no-ai-in-nodejs-core> Accessed: 2026-03-26.
- [28] JabRef Contributors. 2026. AGENTS.md — JabRef. GitHub repository file. <https://github.com/JabRef/jabref/blob/main/AGENTS.md> Accessed: 2026-03-26.
- [29] Jaeger Contributors. 2026. Contributing Guidelines: AI Usage Policy. GitHub repository file. https://github.com/jaegertracing/jaeger/blob/main/CONTRIBUTING_GUIDELINES.md#ai-usage-policy Accessed: 2026-03-26.
- [30] Jaeger Contributors. 2026. Introduce PR limits for new contributors. GitHub Pull Request. <https://github.com/jaegertracing/jaeger/pull/7880> Accessed: 2026-03-26.
- [31] Sushawapak Kancharoendee, Thanat Pichitphanphong, Chanikarn Jongyingyos, Brittany Reid, Raula Gaikovina Kula, Morakot Choetkiertikul, Chaiyong Ragkhitwetsagul, and Thanwadee Sunetnanta. 2025. On categorizing open source software security vulnerability reporting mechanisms on GitHub. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Los Alamitos, CA, USA, 751–756. doi:10.1109/SANER64311.2025.00076
- [32] Syed Mohammad Kashif, Peng Liang, and Amjed Tahir. 2025. On Developers' Self-Declaration of AI-Generated Code: An Analysis of Practices. *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3771937 Online first.
- [33] Kornia Contributors. 2026. Kornia AI and Authorship Policy. GitHub repository file. https://github.com/kornia/kornia/blob/main/AI_POLICY.md Accessed: 2026-03-26.
- [34] Hao Li, Haoxiang Zhang, and Ahmed E Hassan. 2025. The rise of AI teammates in software engineering (SE 3.0): how autonomous coding agents are reshaping software engineering. arXiv preprint arXiv:2507.15003. doi:10.48550/arXiv.2507.15003 [cs.SE]
- [35] Zhixing Li, Yue Yu, Minghui Zhou, Tao Wang, Gang Yin, Long Lan, and Huaimin Wang. 2022. Redundancy, context, and preference: an empirical study of duplicate pull requests in OSS projects. *IEEE Transactions on Software Engineering* 48, 4 (2022), 1461–1476. doi:10.1109/TSE.2020.3018726
- [36] Johan Linäker, Georg J. P. Link, and Kevin Lombard. 2024. Sustaining maintenance labor for healthy open-source software projects through human infrastructure: a maintainer perspective. arXiv preprint arXiv:2408.06723. doi:10.48550/arXiv.2408.06723 [cs.SE]
- [37] llama.cpp Contributors. 2026. AI Usage Policy. GitHub repository file. <https://github.com/ggml-org/llama.cpp/blob/master/CONTRIBUTING.md> Accessed: 2026-03-26.
- [38] llama.cpp Contributors. 2026. Security Policy. GitHub repository file. <https://github.com/ggml-org/llama.cpp/blob/master/SECURITY.md> Accessed: 2026-03-26.
- [39] Matplotlib Contributors. 2026. [PERF] Replace np.column_stack with np.vstack(). GitHub Pull Request. <https://github.com/matplotlib/matplotlib/pull/31132> Accessed: 2026-03-26.
- [40] METR. 2025. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089. doi:10.48550/arXiv.2507.09089 [cs.SE]
- [41] Matthew B. Miles, A. Michael Huberman, and Johnny Saldaña. 2014. *Qualitative Data Analysis: A Methods Sourcebook* (3 ed.). SAGE Publications, Thousand Oaks, CA.
- [42] Dao Sy Duy Minh, Huynh Trung Kiet, Nguyen Lam Phu Quy, Pham Phu Hoa, Tran Chi Nguyen, Nguyen Dinh Ha Duong, and Truong Bao Tran. 2026. Early-Stage Prediction of Review Effort in AI-Generated Pull Requests. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. doi:10.1145/3793302.3793609 arXiv preprint arXiv:2601.00753.
- [43] Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. 2025. Context engineering for AI agents in open-source software. arXiv preprint arXiv:2510.21413. doi:10.48550/arXiv.2510.21413 [cs.SE]
- [44] mpv Contributors. 2026. AI-assisted Contributions. GitHub repository file. <https://github.com/mpv-player/mpv/blob/master/DOCS/contribute.md#ai-assisted-contributions> Accessed: 2026-03-26.
- [45] NetBSD Developers. 2026. NetBSD Commit Guidelines. NetBSD Project Website. <https://www.netbsd.org/developers/commit-guidelines.html> Accessed: 2026-03-26.
- [46] NewPipe Contributors. 2026. NewPipe Contributing: AI policy. GitHub repository file. <https://github.com/TeamNewPipe/NewPipe/blob/dev/.github/CONTRIBUTING.md> Accessed: 2026-03-26.
- [47] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub Copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22)*. Association for Computing Machinery, New York, NY, USA, 1–5. doi:10.1145/3524842.3528470
- [48] pandas Contributors. 2026. Automated Contributions Policy. GitHub repository file. <https://github.com/pandas-dev/pandas/blob/main/doc/source/development/contributing.rst#automated-contributions-policy> Accessed: 2026-03-26.
- [49] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of AI on developer productivity: evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590. doi:10.48550/arXiv.2302.06590 [cs.CY]
- [50] Python Developers. 2026. Generative AI. Python Developer's Guide. <https://devguide.python.org/getting-started/generative-ai/> Accessed: 2026-03-26.
- [51] PyTorch Contributors. 2026. AI-Assisted Development. GitHub repository file. <https://github.com/pytorch/pytorch/blob/main/CONTRIBUTING.md#ai-assisted-development> Accessed: 2026-03-26.
- [52] QEMU Contributors. 2026. Code Provenance: Use of AI-generated content. QEMU Documentation. <https://github.com/qemu/qemu/blob/master/docs/devel/code-provenance.rst> Accessed: 2026-03-26.
- [53] Rapid7 Metasploit Contributors. 2026. Contributing to Metasploit Framework: Vibecoding, AI, and LLM. GitHub repository file. <https://github.com/rapid7/metasploit-framework/blob/master/CONTRIBUTING.md> Accessed: 2026-03-26.
- [54] Robby Russell. 2026. Humans in the Loop. Robby on Rails. <https://robbyonrails.com/articles/2026/01/20/humans-in-the-loop/> Accessed: 2026-03-26.
- [55] Agnia Sergeyuk, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2025. Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology* 178 (2025), 107610. doi:10.1016/j.infsof.2024.107610
- [56] Spec Kit Contributors. 2026. AI Contributions in Spec Kit. GitHub repository file. <https://github.com/github/spec-kit/blob/main/CONTRIBUTING.md#ai-contributions-in-spec-kit> Accessed: 2026-03-26.
- [57] Igor Steinmacher, Marco Aurélio Graciotto Silva, Marco Aurélio Gerosa, and David F Redmiles. 2015. A systematic literature review on the barriers faced by newcomers to open source software projects. *Information and Software Technology* 59 (2015), 67–85. doi:10.1016/j.infsof.2014.11.001
- [58] Daniel Stenberg. 2026. The end of the curl bug-bounty. daniel.haxx.se blog. <https://daniel.haxx.se/blog/2026/01/26/the-end-of-the-curl-bug-bounty/> Accessed: 2026-03-26.
- [59] Emre Süllü, Metehan Saçakçı, and Eray Tüzün. 2024. An empirical analysis of issue template usage in large-scale projects on GitHub. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 117:1–117:28. doi:10.1145/3643673
- [60] tldraw Contributors. 2026. Contributions policy. GitHub Issue. <https://github.com/tldraw/tldraw/issues/7695> Accessed: 2026-03-26.
- [61] Rosalia Tufano, Federica Pepe, Fiorella Zampetti, Antonio Mastropaolo, Ozren Dabić, Massimiliano Di Penta, and Gabriele Bavota. 2026. Developers and generative AI: A study of self-admitted usage in open source projects. *Empirical Software Engineering* 31, 4 (2026), 108. doi:10.1007/s10664-026-10848-w
- [62] typescript-eslint Contributors. 2026. AI Contribution Policy. GitHub repository file. https://github.com/typescript-eslint/typescript-eslint/blob/main/docs/contributing/AI_Contribution_Policy.mdx Accessed: 2026-03-26.
- [63] typescript-eslint Contributors. 2026. Docs: Establish and document expectations around use of AI in contributions. GitHub Issue. <https://github.com/typescript-eslint/typescript-eslint/issues/11416> Accessed: 2026-03-26.
- [64] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, et al. 2025. OpenHands: an open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. OpenReview.net, Singapore, 38 pages. <https://openreview.net/forum?id=QJd3ayDDoF>
- [65] Mairieli Wessel, Joseph Vargovich, Marco A Gerosa, and Christoph Treude. 2023. GitHub Actions: the impact on the pull request process. *Empirical Software Engineering* 28 (2023), 131. doi:10.1007/s10664-023-10369-w
- [66] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE '14)*. ACM,

- New York, NY, USA, 1–10. doi:10.1145/2601248.2601268
- [67] Jiaqi Wu, Lingfeng Bao, Xiaohu Yang, Xin Xia, and Xing Hu. 2024. A large-scale empirical study of open source license usage: practices and challenges. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR '24)*. Association for Computing Machinery, New York, NY, USA, 595–606. doi:10.1145/3643991.3644900
 - [68] Tao Xiao, Youmei Fan, Fabio Calefato, Christoph Treude, Raula Gaikovina Kula, Hideaki Hata, and Sebastian Baltes. 2025. Self-admitted GenAI usage in open-source software. arXiv preprint arXiv:2507.10422. doi:10.48550/arXiv.2507.10422 [cs.SE]
 - [69] Jialiang Xie, Minghui Zhou, and Audris Mockus. 2013. Impact of triage: a study of Mozilla and Gnome. In *2013 ACM / IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM '13*. IEEE, Los Alamitos, CA, USA, 1–10. doi:10.1109/ESEM.2013.62
 - [70] Weiwei Xu, Kai Gao, Hao He, and Minghui Zhou. 2025. LiCoEval: evaluating LLMs on license compliance in code generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering*. IEEE, Los Alamitos, CA, USA, 1665–1677. doi:10.1109/ICSE55347.2025.00052
 - [71] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 50528–50652. doi:10.52202/079017-1601
 - [72] Haruhiko Yoshioka, Takahiro Monno, Haruka Tokumasu, Taiki Wakamatsu, Yuki Ota, Nimmi Rashinika Weeraddana, and Kenichi Matsumoto. 2026. Let's make every pull request meaningful: an empirical analysis of developer and agentic pull requests. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. Association for Computing Machinery, New York, NY, USA, 1–5. <https://arxiv.org/abs/2601.18749> Accepted at MSR 2026 Mining Challenge; preprint available at arXiv. Accessed: 2026-03-26.
 - [73] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2026. A survey on large language models for software engineering. *Science China Information Sciences* 69, 4 (2026), 141102. doi:10.1007/s11432-025-4670-0
 - [74] Songwen Zhao et al. 2025. Is vibe coding safe? Benchmarking vulnerability of agent-generated code in real-world tasks. arXiv preprint arXiv:2512.03262. doi:10.48550/arXiv.2512.03262 [cs.CR]
 - [75] Minghui Zhou, Qingying Chen, Audris Mockus, and Fengguang Wu. 2017. On the scalability of Linux kernel maintainers' work. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*. Association for Computing Machinery, New York, NY, USA, 27–37. doi:10.1145/3106237.3106287
 - [76] Shurui Zhou, Bogdan Vasilescu, and Christian Kästner. 2019. What the fork: a study of inefficient and efficient forking practices in social coding. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 350–361. doi:10.1145/3338906.3338918
 - [77] Zig Contributors. 2025. github: add link to issue template list warning against LLMs. GitHub Commit. <https://github.com/ziglang/zig/commit/d238078ae87f1beb565d42caee01ebd6a7a00d43> Accessed: 2026-03-26.
 - [78] Zig Contributors. 2025. Migrating from GitHub to Codeberg. Zig News. <https://ziglang.org/news/migrating-from-github-to-codeberg/> Accessed: 2026-03-26.